

Chapter 6

Answer 6.1

- a. $p(n, d)$ is equal to the number of features with degree less than d plus the number of features with degree exactly equal to d . The former is $p(n, d - 1)$, by definition. Let $p'(n, d)$ be equal to the number of features with degree exactly equal to d . Therefore,

$$p(n, d) = p(n, d - 1) + p'(n, d),$$

To find $p'(n, d)$, consider we could construct all features with degree exactly d by first constructing all features with degree $\leq d$ for the last $n - 1$ features, and then “augmenting” them by multiplying by x_1 enough times to make the degree exactly equal to d .

For instance, to construct all features of $\{x_1, x_2, x_3\}$ with exactly degree 3, we can construct all features with at most degree 3 from $\{x_2, x_3\}$ to get $\{1, x_2, x_3, x_2^2, x_2x_3, x_3^2, x_2^2x_3, x_2x_3^2, x_3^3\}$. We then multiply in as many x_1 s as necessary to boost each to degree 3 to get $\{x_1^3, x_1^2x_2, x_1^2x_3, x_1x_2^2, x_1x_2x_3, x_1x_3^2, x_2^2x_3, x_2x_3^2, x_3^3\}$. And, we end up with all features with exactly degree 3 for the original set $\{x_1, x_2, x_3\}$.

Thus, $p'(n, d) = p(n - 1, d)$ and we have the recurrence relation

$$p(n, d) = p(n, d - 1) + p(n - 1, d)$$

where

$$p(0, d) = p(n, 0) = 1$$

which is the recurrence relationship for the binomial coefficients. Thus

$$p(n, d) = \binom{n + d}{d}$$

which can be verified:

$$\begin{aligned} p(n, d - 1) + p(n - 1, d) &= \binom{n + d - 1}{d - 1} + \binom{n + d - 1}{d} \\ &= \frac{(n + d - 1)!}{n!(d - 1)!} + \frac{(n + d - 1)!}{(n - 1)!d!} \\ &= \frac{d(n + d - 1)!}{n!d!} + \frac{n(n + d - 1)!}{n!d!} \\ &= \frac{(n + d)(n + d - 1)!}{n!d!} \\ &= \frac{(n + d)!}{n!d!} = \binom{n + d}{d}. \end{aligned}$$

- b. Here are two solutions in Python, using the numpy libraries for matrices. Solutions in other languages would be similar.

The code below follows the construction above:

```
import numpy as np
```

```
def polyfeat(X, d):
    m, n = X.shape
```

```

if d==0 or n==0:
    return np.ones((m,1)),[0]
# get features without x_1
Xd,degs = polyfeat(X[:,1:],d)
# multiply each by enough copies of x_1 to make degree d
for i,dd in enumerate(degs):
    Xd[:,i] *= X[:,0]**(d-dd)
# get all features with degree < d
Xdless,lessdegs = polyfeat(X,d-1)
# return both together
return np.hstack((Xdless,Xd)),lessdegs+([d]*len(degs))

```

It returns the degree of each feature (column), along with the answer, to help with the construction. Below is a more compact, efficient version using itertools.

```

import numpy as np
from itertools import chain, combinations_with_replacement

def powersupto(s,d):
    return chain.from_iterable(
        combinations_with_replacement(s,r)
        for r in range(d+1))

def polyfeat(X,d):
    n = X.shape[1]
    return np.hstack(tuple(
        [np.prod(X[:,ind],axis=1,keepdims=True)
         for ind in powersupto(range(n),d)]))

```

Answer 6.2

- a. If $\lambda = 0$, ridge regression is the same as linear least squares without regularization. So we get the

same answer as for Exercise 4.2: $\vec{w} = A^{-1}\vec{c} = \begin{bmatrix} 2.18 \\ -0.21 \\ 0.41 \\ 1.34 \end{bmatrix}$.

- b. If $\lambda = 10$ and we penalize all of the weights, we calculate the same A and $\vec{c}$ as in Exercise 4.2:

$$A = X^T X = \begin{bmatrix} 4.00 & 8.00 & -0.10 & 1.30 \\ 8.00 & 42.74 & -4.13 & -1.61 \\ -0.10 & -4.13 & 4.25 & 2.97 \\ 1.30 & -1.61 & 2.97 & 4.35 \end{bmatrix}$$

$$\vec{c} = X^T Y = \begin{bmatrix} 8.70 \\ 4.41 \\ 6.38 \\ 10.22 \end{bmatrix}.$$

Thus

$$\vec{w} = (\mathbf{A} + 10\mathbf{I})^{-1} \vec{c} = \begin{bmatrix} 0.54 \\ 0.05 \\ 0.34 \\ 0.60 \end{bmatrix}$$

c. If $\lambda = 10$ but we do not penalize the intercept term, we calculate

$$\vec{w} = \left(\mathbf{A} + 10 \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \right)^{-1} \vec{c} = \begin{bmatrix} 2.61 \\ -0.28 \\ 0.31 \\ 0.38 \end{bmatrix}$$

Answer 6.3

$$\begin{aligned} \vec{0} &= \nabla_{\vec{w}} \left(\sum_{i=1}^m (y_i - \vec{x}_i^\top \vec{w})^2 + \sum_{j=1}^n \lambda_j w_j^2 \right) \\ &= \nabla_{\vec{w}} \left(\sum_{i=1}^m (y_i - \vec{x}_i^\top \vec{w})^2 + \vec{w}^\top \underbrace{\begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}}_{\triangleq \mathbf{\Lambda}} \vec{w} \right) \\ &= -2 \sum_{i=1}^m \vec{x}_i (y_i - \vec{x}_i^\top \vec{w}) + 2\mathbf{\Lambda} \vec{w} \\ \left(\sum_{i=1}^m \vec{x}_i y_i \right) &= \left(\sum_{i=1}^m \vec{x}_i \vec{x}_i^\top + \mathbf{\Lambda} \right) \vec{w} \\ \vec{w} &= (\mathbf{A} + \mathbf{\Lambda})^{-1} \vec{c} \end{aligned}$$

Answer 6.4

- Cannot be fit using linear least-squares regression
- Can be fit: $\phi(\vec{x}) = \begin{bmatrix} e^{x_1} \\ e^{x_2} \end{bmatrix}$
- Cannot be fit using linear least-squares regression
- Can be fit: $\phi(\vec{x}) = \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ x_1 \\ x_2 \end{bmatrix}$
- Can be fit: $\phi(\vec{x}) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$
- Can be fit: $\phi(\vec{x}) = \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}$
- Can be fit: $\phi(\vec{x}) = \begin{bmatrix} x_1^2 \\ x_2^2 \\ x_1 x_2 \\ 1 \end{bmatrix}$

- h. Can be fit: $\phi(\vec{x}) = \begin{bmatrix} x_1^2 + x_2^2 \\ 1 \end{bmatrix}$
- i. Not linear in w_2, w_3 so cannot be fit as given. However, if w_2 and w_3 have no other constraints, they are redundant, and we can set $w_3 = 1$ and continue with linear least-squares regression as normal.
- j. Cannot be fit just by feature mapping. With feature mapping $\phi(\vec{x}) = [x_1]$, the x_2 term is ignored. However, if we note that the error can be rewritten

$$\begin{aligned} y - f(\vec{x}) &= y - w_1 x_1 - x_2 \\ &= \underbrace{(y - x_2)}_{\text{new target } y} - \underbrace{w_1 x_1}_{\text{new } f \text{ form}} \end{aligned}$$

we can redefine y as well (to be the old y minus x_2) and then fit using ϕ and the new y values.

Answer 6.5

- The orange curve is the parameter training error because it is monotonically increasing with the regularization strength.
- The blue curve is the validation error because the chosen value for λ is at its minimum.
- The green curve is the testing error because it is the only one left. (It could, in theory, have any shape.)
- $\lambda = 20$ (approximately) gives the minimum testing error and would therefore be ideal.
- We do not know the testing data at training time, so we cannot use the results of the testing data to select λ .

Answer 6.6

Note that

$$\frac{\partial R(\vec{w})}{\partial w_j} = p |w_j|^{p-1} \text{sgn}(w_j)$$

and thus

$$\nabla_{\vec{w}} R(\vec{w}) = \begin{bmatrix} p |w_1|^{p-1} \text{sgn}(w_1) \\ p |w_2|^{p-1} \text{sgn}(w_2) \\ \vdots \\ p |w_n|^{p-1} \text{sgn}(w_n) \end{bmatrix}$$

- a. The derivation follows that of ridge regression's gradient closely:

$$\begin{aligned} \nabla_{\vec{w}} L &= \nabla_{\vec{w}} \left(\sum_{i=1}^m (y_i - \vec{x}_i^T \vec{w})^2 + \lambda R(\vec{w}) \right) \\ &= - \left(\sum_{i=1}^m \vec{x}_i y_i \right) + \left(\sum_{i=1}^m \vec{x}_i \vec{x}_i^T \right) \vec{w} + \lambda \nabla_{\vec{w}} R(\vec{w}) \quad \text{using the results from the text and the derivation above} \\ &= \mathbf{X}^T \mathbf{X} \vec{w} - \mathbf{X}^T \mathbf{Y} + \lambda \nabla_{\vec{w}} R(\vec{w}) . \end{aligned}$$

Thus the update rule is

$$\vec{w} \leftarrow \vec{w} - \eta \mathbf{X}^T \mathbf{X} \vec{w} + \eta \mathbf{X}^T \mathbf{Y} - \eta \lambda \nabla_{\vec{w}} R(\vec{w}) .$$

b. The derivation follows that of logistic regression's gradient closely:

$$\begin{aligned}\nabla_{\vec{w}} L &= \nabla_{\vec{w}} \left(\sum_{i=1}^m -\ln \sigma(y_i \vec{x}_i^\top \vec{w}) + \lambda R(\vec{w}) \right) \\ &= - \left(\sum_{i=1}^m (1 - \sigma(y_i \vec{x}_i^\top \vec{w})) y_i \vec{x}_i \right) + \lambda \nabla_{\vec{w}} R(\vec{w}) \quad \text{using the results from the text and the derivation above}\end{aligned}$$

Thus the update rule is

$$\vec{w} \leftarrow \vec{w} + \eta \sum_{i=1}^m (1 - \sigma(y_i \vec{x}_i^\top \vec{w})) y_i \vec{x}_i - \eta \lambda \nabla_{\vec{w}} R(\vec{w}).$$

c. When $p \leq 1$, the derivative and gradient of R are not defined whenever any $w_j = 0$. Further, the gradient does not decrease near $w_j = 0$, but either remains constant ($p = 1$) or increases ($p < 1$) as w_j approaches 0. This means gradient descent will not converge because the closer w_j gets to zero, the faster it will be "propelled" across 0.

Answer 6.7

The calculated weight vector for the i th fold is

$$\vec{w}_{-i} = (\mathbf{X}_{-i}^\top \mathbf{X}_{-i} + \lambda \mathbf{I})^{-1} \mathbf{X}_{-i}^\top \mathbf{Y}_{-i}$$

and thus the prediction for the validation point at the i th fold is

$$\hat{y}_{i,-i} = \vec{x}_i^\top (\mathbf{X}_{-i}^\top \mathbf{X}_{-i} + \lambda \mathbf{I})^{-1} \mathbf{X}_{-i}^\top \mathbf{Y}_{-i}$$

Therefore the total leave-one-out cross-validation error is

$$\begin{aligned}\sum_{i=1}^m (y_i - \hat{y}_{i,-i})^2 &= \sum_{i=1}^m \left(y_i - \vec{x}_i^\top (\mathbf{X}_{-i}^\top \mathbf{X}_{-i} + \lambda \mathbf{I})^{-1} \mathbf{X}_{-i}^\top \mathbf{Y}_{-i} \right)^2 \\ &= \sum_{i=1}^m \left(y_i - \vec{x}_i^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I} - \vec{x}_i \vec{x}_i^\top)^{-1} (\mathbf{X}^\top \mathbf{Y} - \vec{x}_i y_i) \right)^2\end{aligned}$$

letting $\mathbf{B} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1}$,

$$\begin{aligned}&= \sum_{i=1}^m \left(y_i - \vec{x}_i^\top \left(\mathbf{B} + \mathbf{B} \frac{\vec{x}_i \vec{x}_i^\top}{1 - \vec{x}_i^\top \mathbf{B} \vec{x}_i} \mathbf{B} \right) (\mathbf{X}^\top \mathbf{Y} - \vec{x}_i y_i) \right)^2 \quad \text{using the Woodbury identity} \\ &= \sum_{i=1}^m \left(y_i - \vec{x}_i^\top \mathbf{B} \mathbf{X}^\top \mathbf{Y} + \vec{x}_i^\top \mathbf{B} \vec{x}_i y_i - \vec{x}_i^\top \mathbf{B} \frac{\vec{x}_i \vec{x}_i^\top}{1 - \vec{x}_i^\top \mathbf{B} \vec{x}_i} \mathbf{B} \mathbf{X}^\top \mathbf{Y} + \vec{x}_i^\top \mathbf{B} \frac{\vec{x}_i \vec{x}_i^\top}{1 - \vec{x}_i^\top \mathbf{B} \vec{x}_i} \mathbf{B} \mathbf{X} \vec{x}_i y_i \right)^2\end{aligned}$$

noting $\hat{y}_i = \vec{x}_i^\top \mathbf{B} \mathbf{X}^\top \mathbf{Y}$

and defining $h_i = \vec{x}_i^\top \mathbf{B} \vec{x}_i$,

$$\begin{aligned}&= \sum_{i=1}^m \left(y_i - \hat{y}_i + h_i y_i - \frac{h_i \hat{y}_i}{1 - h_i} + \frac{h_i^2 y_i}{1 - h_i} \right)^2 \\ &= \sum_{i=1}^m \left(\frac{y_i - y_i h_i - \hat{y}_i + \hat{y}_i h_i + h_i y_i - h_i^2 y_i - h_i \hat{y}_i + h_i^2 y_i}{1 - h_i} \right)^2 \\ &= \sum_{i=1}^m \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2\end{aligned}$$

Answer 6.8

a. A circle classifier has the form

$$\tilde{f}(\vec{x}) = c \times (x_1^2 + x_2^2)^2 - d \leq 0$$

where the sign of c and d specifies whether the inside or outside is the positive class and the radius of the circle is equal to $\sqrt{\frac{d}{c}}$.

Expanded out this gives a discriminant of the form

$$\tilde{f}(\vec{x}) = cx_1^2 + cx_2^2 - d$$

which is a specific case of the general linear classifier in part a:

$$\tilde{f}(\vec{x}) = \vec{w}^\top \phi(\vec{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2$$

where $w_0 = -d$, $w_1 = w_2 = w_4 = 0$, and $w_3 = w_5 = c$.

b. From above, we need two features:

$$\phi(\vec{x}) = \begin{bmatrix} 1 \\ x_1^2 + x_2^2 \end{bmatrix}.$$

This gives us a discriminant of the form

$$\tilde{f}(\vec{x}) = w_0 + w_1(x_1^2 + x_2^2)$$

As with all linear discriminants, there is a degree of redundancy. All decision boundaries using this feature mapping can be written as

$$\begin{aligned} w_0 + w_1(x_1^2 + x_2^2) &= 0 \\ \frac{w_0}{w_1} + (x_1^2 + x_2^2) &= 0 \\ x_1^2 + x_2^2 &= -\frac{w_0}{w_1} \end{aligned}$$

which is the equation for a circle centered at the origin, regardless of the values of w_0 and w_1 . (If w_0 and w_1 have the same sign, then there will be no decision boundary and the entire plane will have the same prediction — which is a form of a very degenerate circle.)

c. A general circle classifier has the form

$$\tilde{f}(\vec{x}) = c((x_1 - a)^2 + (x_2 - b)^2) - d \leq 0$$

This is not linear in the parameters a and b . However, we can rework the equation as

$$\begin{aligned} \tilde{f}(\vec{x}) &= (ca^2 + cb^2 - d) - 2acx_1 - 2bcx_2 + c(x_1^2 + x_2^2) \\ &= w_0 + w_1x_1 + w_2x_2 + w_3(x_1^2 + x_2^2) \end{aligned}$$

for

$$w_0 = ca^2 + cb^2 - d \quad w_1 = -2ac \quad w_2 = -2bc \quad w_3 = c$$

which has the reverse mapping of

$$a = \frac{-w_1}{2w_3} \quad b = \frac{-w_2}{2w_3} \quad c = w_3d = -\frac{w_1^2 + w_2^2}{4w_3} - w_0$$

Thus there is a one-to-one mapping between values of a, b, c, d and those of w_0, w_1, w_2, w_3 and all $\vec{w}$ in the form

$$\tilde{f}(\vec{x}) = \vec{w}^T \phi(\vec{x}) \quad \text{where} \quad \phi(\vec{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}$$

correspond to circle decision boundaries.

- d. Assume there were a feature mapping such that all classifiers corresponded to a decision boundary of a circle where the inside was the positive class and the outside was the negative class. Let $\vec{w}$ be any weight vector. It corresponds to such a decision boundary. The weight vector $-\vec{w}$ corresponds to the same decision boundary because $\vec{w}^T \phi(\vec{x}) = 0$ is the same as $-\vec{w}^T \phi(\vec{x}) = 0$. However, the discriminant is exactly negation. Therefore, the labeling of positive and negative classes will be the reverse for $-\vec{w}$ and the outside will be the positive class and the inside the negative class. However, $-\vec{w}$ is a valid weight vector. Thus, the mapping does not prevent decision circles where the inside is negative and the outside is positive.
- e. Consider axis-aligned ellipses centered at the origin (although the same holds for those not so constrained). They take the form

$$ax_1^2 + bx_2^2 - 1 = 0$$

where both a and b are positive. While this is linear in the parameters, a and b , there are constraints on the parameters which are not enforced in (general) linear classification.

We can introduce an extra parameter to try to take advantage of the extra parameter of redundancy in linear discriminants:

$$ax_1^2 + bx_2^2 - d = 0.$$

(The radii are now $\sqrt{\frac{d}{a}}$ and $\sqrt{\frac{d}{b}}$ instead of $\sqrt{\frac{1}{a}}$ and $\sqrt{\frac{1}{b}}$.) However, we now have the constraint that d , a , and b have the same sign.

We can use their squares instead to force all coefficients to have the right sign:

$$a^2x_1^2 + b^2x_2^2 - d^2 = 0.$$

However, this is not linear in the parameters (a , b , and d).

In general, we have two degrees of freedom and need to allow them to vary independently, but we cannot force them to have the same sign without imposing a non-linear transform of the parameters.